

- RAG Learning

## 4.9 Communication & Networking

Computer Science — AQA A-Level (7517) — Paper 2: Written Theory

Topic 4.9 is by far the largest theory unit in the entire specification. It weaves together physics (signals, attenuation, media), engineering (protocols, layered stacks, encapsulation), and a good deal of binary arithmetic (IP addressing, subnets, CIDR). The examiner rewards candidates who can keep these layers separate in their head: a question about MAC addresses lives at a different layer from a question about DNS, even though both can show up in the same scenario. Work from the bottom up in your revision — signals and media, then frames and switches, then packets and routers, then transport (TCP/UDP), then application (HTTP, DNS, SMTP). Once the layer cake is drawn on a mental napkin, every other concept in this topic slots into one of its layers.

**Exam Tip:** When a networking question spans multiple layers (a common 8-mark device), explicitly state which layer you are operating at before each move. "At the data-link layer, the switch forwards the frame using the destination MAC address. At the network layer, the router forwards the packet using the destination IP." Examiners mark by layer; signalling the layer is worth marks even before you get the detail right.

### 4.9.1 Communication Methods

#### Serial vs parallel transmission

**Serial transmission** — bits are sent one after another down a single data line. **Parallel transmission** — several bits (typically 8, 16, 32 or 64) are sent simultaneously down multiple parallel data lines, one wire per bit.

Parallel is intuitively faster because  $n$  wires carry  $n$  bits per clock tick. In practice modern high-speed interfaces are almost all serial: SATA replaced PATA, PCIe replaced PCI, USB replaced the parallel printer port. The two reasons are physical.

- **Skew.** Parallel bits that left the transmitter together do not necessarily arrive together. Tiny differences in wire length, capacitance and impedance mean that one line's bit arrives a few picoseconds before another's. At low clock rates this is negligible; at gigabit-plus rates the spread of arrival times exceeds the bit period and the receiver cannot tell which set of bits belongs together. Parallel links therefore cannot be clocked arbitrarily fast; serial links can.
- **Crosstalk.** Adjacent parallel wires behave as coupled inductors and capacitors: a rising edge on one line induces noise on its neighbours. The problem worsens as wires get closer (miniaturisation) and as data rates rise. Serial links have fewer adjacent wires and can be shielded more aggressively (twisted pair, differential signalling).

Parallel still wins in very short, controlled environments — the internal data bus of a CPU, the address and data buses on a motherboard — where distance is millimetres and clock can be kept modest. For anything longer than a few centimetres, modern design is serial with differential signalling.

## Synchronous vs asynchronous transmission

**Synchronous transmission** — sender and receiver share a common clock signal (either a physical clock line or a clock embedded in the data). Bits are sent as a continuous stream with no per-character delimiters. **Asynchronous transmission** — no shared clock; each byte or character is framed by a start bit and one or more stop bits that the receiver uses to resynchronise its local clock at every character.

Asynchronous transmission is historically what a UART (Universal Asynchronous Receiver-Transmitter) does, as used by classic RS-232 serial ports: the idle line sits high, a falling edge (start bit, logic 0) tells the receiver "a character is coming, start sampling at the agreed baud rate", then 7 or 8 data bits follow, then 1 or 2 stop bits (logic 1) return the line to idle. The overhead per byte is 2–3 bits — roughly 20–25% — which is acceptable for keyboard-speed data but wasteful for bulk transfer.

Synchronous transmission amortises overhead across much larger frames (thousands of bytes). Ethernet, USB 2.0+ and SATA are all effectively synchronous, using embedded clock-recovery schemes (8b/10b or 64b/66b line coding) so that the clock is recoverable from the data transitions themselves — no separate clock line is needed.

## Baud rate, bit rate and bandwidth

**Baud rate** — the number of *signalling events* (symbol changes) per second on the medium, measured in baud. **Bit rate** — the number of *bits* transmitted per second, measured in bits/s. **Bandwidth** — in networking, the maximum data rate a link can sustain (bit/s); in signal theory, the width of the frequency range the medium supports (Hz).

Baud rate and bit rate are equal only when each symbol encodes exactly one bit. Modern modulation schemes pack multiple bits per symbol: a constellation with four phase positions (QPSK) encodes 2 bits per symbol, so bit rate = 2 × baud. A 256-QAM constellation encodes 8 bits per symbol, so bit rate = 8 × baud. The absolute ceiling is given by the **Shannon–Hartley theorem**: channel capacity  $C = B \cdot \log_2(1 + \text{SNR})$ , where  $B$  is analogue bandwidth in hertz and SNR is the signal-to-noise ratio. Real engineering pushes towards this ceiling by using denser modulation until the noise floor defeats it.

## Latency, attenuation, noise

- **Latency** is the time delay between a bit being sent and received. Caused by propagation delay (speed of light in the medium), serialisation delay (time to push bits onto the wire at the given rate), queueing delay at each router, and processing delay at each hop. A transatlantic fibre has around 30–60 ms round-trip time, which is a hard physical floor — no amount of engineering reduces it below the speed-of-light budget.
- **Attenuation** is the gradual loss of signal strength over distance. Copper attenuates more at higher frequencies; fibre attenuates very little (around 0.2 dB/km at 1550 nm). Links beyond their attenuation budget need repeaters/amplifiers.
- **Noise** is unwanted random signal added to the data — thermal noise, electromagnetic interference from nearby equipment, crosstalk from other wires. Noise raises the bit-error rate; fighting noise is the job of shielding, twisted pairs, differential signalling and error-detecting/correcting codes.

## Error detection

- **Parity.** A single extra bit is appended to each byte such that the total number of 1s is even (even parity) or odd (odd parity). Detects any odd number of bit errors; misses even-numbered ones. Cheap and historical — RS-232 memory.
- **Checksum.** The sender sums the data bytes (modulo some fixed N), appends the sum, and the receiver recomputes. Catches more errors than parity but still misses patterns that sum to the same value.
- **Cyclic Redundancy Check (CRC).** The sender treats the data as a large binary polynomial, divides by a fixed generator polynomial, and appends the remainder. Extremely good at catching burst errors — Ethernet uses CRC-32, which catches all burst errors up to 32 bits and detects larger bursts with probability  $1-2^{-32}$ .

## Error correction — Hamming code

Parity, checksum and CRC only *detect* errors; the receiver must ask for retransmission. **Forward error correction** goes further: redundancy bits are added so the receiver can reconstruct the original without retransmission. Hamming(7,4) encodes 4 data bits as 7 transmitted bits (three parity bits interleaved with the data) and can correct any single-bit error. Used where retransmission is expensive or impossible — deep-space links (Voyager), ECC RAM, flash storage.

## 4.9.2 Communication Basics

### Simplex, half-duplex, full-duplex

| Mode        | Direction       | Simultaneous?        | Example                                       |
|-------------|-----------------|----------------------|-----------------------------------------------|
| Simplex     | One-way only    | N/A                  | Broadcast TV/radio, keyboard to PC            |
| Half-duplex | Both directions | No — one at a time   | Walkie-talkies ("over"), classic Ethernet hub |
| Full-duplex | Both directions | Yes — simultaneously | Telephone, switched Ethernet                  |

## Broadcast, unicast, multicast

- **Unicast** — one sender, one recipient. The overwhelming majority of internet traffic (a web page served to your browser).
- **Broadcast** — one sender, every node on the network. ARP requests are broadcast ("who has IP 192.168.1.1?"). Expensive at scale, so broadcast is typically confined to a single LAN segment.
- **Multicast** — one sender, a subscribed group of recipients. Used for IPTV, live-stream sports distribution inside an ISP, and routing-protocol control traffic (OSPF hellos). Sits between unicast and broadcast in cost.

## Packet switching vs circuit switching

**Circuit switching** — a dedicated end-to-end path is reserved for the duration of a conversation; bandwidth is guaranteed. Classic example: the traditional telephone network (PSTN). **Packet switching** — data is chopped into packets, each labelled with a destination address, and each packet is routed independently across a shared network; no path is reserved in advance.

| Property                      | Circuit switching                             | Packet switching                                   |
|-------------------------------|-----------------------------------------------|----------------------------------------------------|
| Path setup                    | Explicit call setup; a fixed route held open  | No setup; packets routed independently (datagrams) |
| Bandwidth                     | Guaranteed for the circuit                    | Best-effort; shared across many flows              |
| Efficiency                    | Low — idle circuit still consumes capacity    | High — idle senders consume nothing                |
| Resilience to link failure    | Poor — call drops if a link on the path fails | Good — packets reroute around failures             |
| Jitter/latency predictability | Very predictable                              | Variable (queueing)                                |

|          |                         |                                           |
|----------|-------------------------|-------------------------------------------|
| Use case | Historical voice (PSTN) | The internet; modern voice (VoIP over IP) |
|----------|-------------------------|-------------------------------------------|

Packet switching has two flavours. In a pure **datagram** service (IP), each packet stands alone and can take a different route to its peer. In a **virtual-circuit** service (ATM, MPLS, some legacy X.25), a logical path is signalled before data flows, all subsequent packets follow that label-switched path, but the underlying network is still statistically multiplexed rather than reserving physical capacity. Virtual circuits deliver predictable ordering and latency without the bandwidth waste of a true circuit.

**Packet vs circuit switching** — packet switching statistically multiplexes many flows over shared links; circuit switching reserves end-to-end capacity for the duration of a conversation. The internet chose packet switching because it scales efficiently and survives partial failures; voice networks historically chose circuit switching because it delivered predictable voice quality — a choice that has now been reversed as VoIP matured.

## 4.9.3 Networks — Types and Topologies

### Network types by scale

**LAN (Local Area Network)** — a network confined to a single site (a home, a school, an office floor or building), typically owned and administered by one organisation. Built from Ethernet and Wi-Fi. High bandwidth (1–100 Gbps), low latency (<1 ms), private address space.

**WAN (Wide Area Network)** — a network spanning a wide geographical area (a country or globe), usually leased from telecommunications carriers rather than owned by one organisation. Lower bandwidth per link than LAN and higher latency. The internet is the world's biggest WAN.

- **PAN (Personal Area Network)** — device-to-device within a few metres, typically Bluetooth or Zigbee (phone to earbuds, fitness tracker to watch).
- **MAN (Metropolitan Area Network)** — spans a city, often a university or municipal fibre ring connecting multiple LANs.

## Topologies

| Topology        | Structure                                                                                           | Strengths                                                                                | Weaknesses                                                                                                                     |
|-----------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Bus             | All nodes share one common backbone cable                                                           | Cheap; little cabling                                                                    | Single-point failure (cable break = whole LAN down); collisions require CSMA/CD; obsolete for Ethernet                         |
| Star (physical) | All nodes cabled back to a central switch                                                           | Dominant modern LAN; switch failure isolates one port; cable fault affects only one host | Central switch is a single point of failure; more cabling than bus                                                             |
| Star (logical)  | A physical star in which traffic is logically broadcast by a hub                                    | Simple                                                                                   | Hub shares bandwidth — replaced by switches years ago                                                                          |
| Ring            | Each node connects to exactly two neighbours forming a closed loop; token passing arbitrates access | Deterministic access time; no collisions                                                 | Single cable fault breaks the ring (dual-counter-rotating rings mitigate); IBM Token Ring obsolete, SONET/SDH still uses rings |
| Mesh (full)     | Every node connected to every other node                                                            | Maximum redundancy and fault tolerance                                                   | $O(n^2)$ links — impractical beyond a handful of nodes                                                                         |
| Mesh (partial)  | Critical nodes have multiple paths; others don't                                                    | Practical redundancy; used in WAN backbones and Wi-Fi mesh (e.g. home extenders)         | More complex routing; more expensive than star                                                                                 |

**Physical vs logical topology.** A modern Ethernet LAN is *physically* a star (each host plugs into a central switch) but may be *logically* a bus from the point of view of a protocol analyser: the original 10BASE-T standard broadcast every frame on every port via a hub. Switched Ethernet broke that logical bus by forwarding each frame only to the port on which the destination MAC lives. Always answer exam questions with both views: "physically a star, logically a switched star — frames are not broadcast, they are forwarded to the correct port."

## Wireless networks — Wi-Fi standards

Wi-Fi is governed by IEEE 802.11 standards. You should recognise the generation names.

| IEEE     | Wi-Fi name   | Max throughput | Band               |
|----------|--------------|----------------|--------------------|
| 802.11b  | —            | 11 Mbps        | 2.4 GHz            |
| 802.11g  | —            | 54 Mbps        | 2.4 GHz            |
| 802.11n  | Wi-Fi 4      | 600 Mbps       | 2.4 and 5 GHz      |
| 802.11ac | Wi-Fi 5      | ~3.5 Gbps      | 5 GHz              |
| 802.11ax | Wi-Fi 6 / 6E | ~9.6 Gbps      | 2.4, 5, 6 GHz (6E) |
| 802.11be | Wi-Fi 7      | ~46 Gbps       | 2.4, 5, 6 GHz      |

## 4.9.4 The Internet and its Uses

The internet is best described as a **WAN of WANs** — a global, interconnected collection of autonomous networks that agree to exchange traffic using the TCP/IP protocol suite. No single entity owns it; its routing emerges from bilateral and multilateral **peering and transit** agreements between Autonomous Systems (AS) — each AS is an ISP, university, tech giant, or similar organisation with its own routing policy and an AS number.

### ISPs and the backbone

An **Internet Service Provider (ISP)** sells connectivity. Residential ISPs (BT, Virgin Media, Sky) deliver to homes; they themselves buy upstream transit from larger **Tier 1** carriers (Level 3/Lumen, NTT, Tata, Cogent, Deutsche Telekom) who peer globally without paying anyone for transit. The **internet backbone** is the informal term for the mesh of high-capacity fibre links between Tier 1 networks, typically landing at major **Internet Exchange Points (IXPs)** such as LINX (London), DE-CIX (Frankfurt) and AMS-IX (Amsterdam) where hundreds of networks physically interconnect.

## URL anatomy

```
scheme://user:pass@host:port/path?query#fragment
```

```
https://www.example.co.uk:443/products/item?id=42&sort=price#reviews
```

|        |      |      |      |       |          |
|--------|------|------|------|-------|----------|
|        |      |      |      |       |          |
| scheme | host | port | path | query | fragment |

- **Scheme** — the protocol ( `http` , `https` , `ftp` , `mailto` , `ssh` ).
- **Host** — a DNS name or an IP literal.
- **Port** — optional; defaults to 80 for http, 443 for https.
- **Path** — resource path on the server.
- **Query string** — key=value pairs passed to the server.
- **Fragment** — client-side only; the browser scrolls to this anchor, never sent to the server.

## Internet governance — ICANN, IANA, RIRs

- **ICANN** (Internet Corporation for Assigned Names and Numbers) — the non-profit which overall coordinates the DNS root and IP allocation policy.
- **IANA** (Internet Assigned Numbers Authority) — a function operated by ICANN: allocates IP blocks to the RIRs and maintains protocol-number registries (port numbers, ICMP types).
- **Regional Internet Registries (RIRs)** — five regional bodies that allocate IPs to ISPs and LIRs: RIPE NCC (Europe/Middle East), ARIN (North America), APNIC (Asia-Pacific), LACNIC (Latin America), AFRINIC (Africa).

## Content Delivery Networks (CDNs)

A CDN (Cloudflare, Akamai, AWS CloudFront, Fastly) is a geographically distributed cache network that mirrors origin content close to end users. Benefits: lower latency (a London user hits a London CDN edge, not a Virginia origin); reduced origin load; DDoS absorption; TLS termination at the edge. Modern web relies heavily on CDNs — a significant outage at a major CDN (Cloudflare June 2022, Fastly June 2021) causes widespread internet disruption.

## 4.9.5 Internet Structure — DNS

**DNS (Domain Name System)** — the distributed, hierarchical database that translates human-readable domain names ( `www.example.com` ) into IP addresses and other records. Often described as the "phone book of the internet".

### DNS hierarchy

The DNS namespace is a tree, read right-to-left:



- **Root servers** — 13 logical root nameservers (A through M), mirrored to hundreds of physical sites via anycast. They know where the TLD servers are.
- **TLD servers** — one set per TLD. `.com` is operated by Verisign; `.uk` is operated by Nominet. They know where each second-level domain's authoritative servers are.
- **Authoritative name servers** — run by the domain owner (or their registrar/DNS-hosting provider). Contain the actual resource records for a zone.

### Recursive vs iterative resolution

When your browser resolves `www.example.com`, the OS asks a **recursive resolver** (typically run by your ISP, or 1.1.1.1/8.8.8.8). The resolver does the work on your behalf. *Recursive* means "please give me the final answer"; *iterative* is the step-by-step walk the resolver performs across the DNS tree.

DNS LOOKUP OF `www.example.com` (uncached):

```
Client → Resolver:    "Give me A record for www.example.com" (recursive)
Resolver → Root:      "www.example.com ?"
Root → Resolver:      "I don't know, ask the .com TLD at 192.5.6.30"
Resolver → .com TLD:  "www.example.com ?"
.com TLD → Resolver:  "ask example.com's NS at ns1.example.com (A.B.C.D)"
Resolver → ns1.example: "www.example.com ?"
ns1.example → Resolver: "A record = 93.184.216.34"    <- authoritative
Resolver → Client:    "93.184.216.34"
Resolver caches for the TTL (e.g. 3600 s) so the next lookup is one hop.
```

## Caching

DNS would collapse without caching. Every record carries a TTL (time-to-live) in seconds; resolvers must not serve the record beyond its TTL. Browser, OS stub-resolver, and recursive-resolver caches all honour it. Operators lower TTLs (to 60 s, say) shortly before a planned migration so that the change propagates quickly.

## Common record types

| Type  | Purpose                                                                |
|-------|------------------------------------------------------------------------|
| A     | IPv4 address for a name                                                |
| AAAA  | IPv6 address ("quad-A")                                                |
| CNAME | Canonical name — alias one name to another                             |
| MX    | Mail exchanger — where to deliver email for the domain (with priority) |
| NS    | Nameserver — who is authoritative for this zone                        |

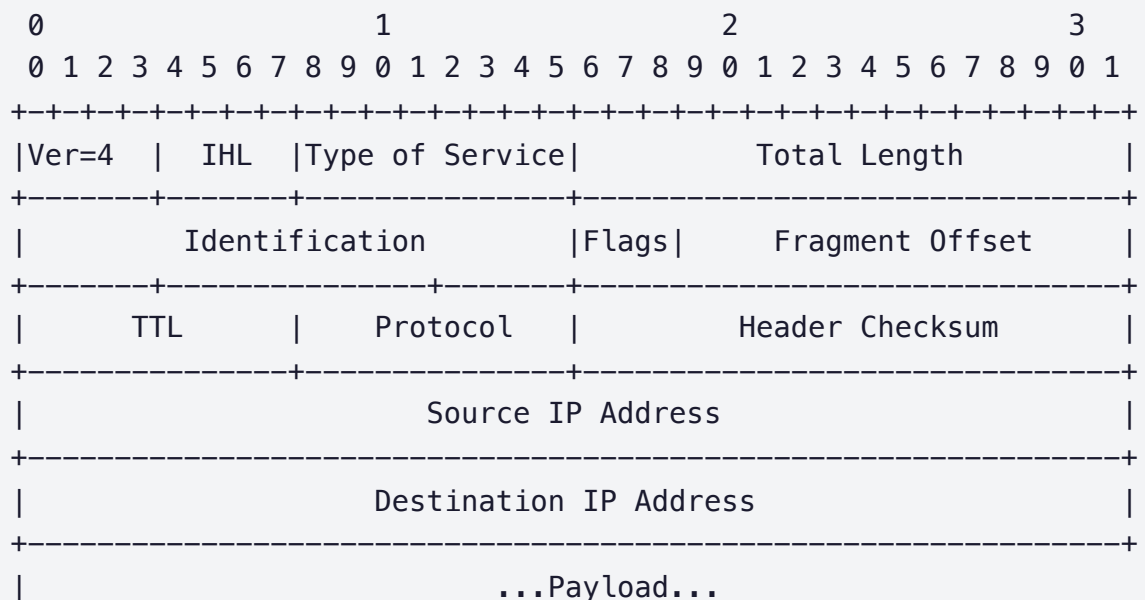
|     |                                                                   |
|-----|-------------------------------------------------------------------|
| TXT | Arbitrary text — SPF, DKIM, domain-verification tokens            |
| PTR | Reverse — IP back to name (in the <code>in-addr.arpa</code> zone) |
| SOA | Start of authority — zone metadata (serial, refresh, expire)      |

**Exam Tip:** DNS resolution is a favourite 6-mark question. Always mention: (1) the recursive resolver, (2) root → TLD → authoritative walk, (3) caching with TTL, (4) the returned IP is used to open a TCP connection — DNS itself is almost always carried over UDP on port 53. Examiners award a mark per bullet; missing any of the four caps your mark.

## 4.9.6 Packet Switching and Routing

### Packet structure

An IPv4 packet consists of a header (minimum 20 bytes) followed by a payload. Key header fields you should know:



- **Source / destination IP** — 32 bits each; who sent it, where it is going.

- **TTL (Time to Live)** — decremented by each router. When it hits zero, the packet is discarded and an ICMP Time-Exceeded message is sent back. Prevents packets looping forever if a routing table is misconfigured.
- **Protocol** — what's inside the payload (6 = TCP, 17 = UDP, 1 = ICMP).
- **Header checksum** — protects the header only. The payload is protected by TCP/UDP checksums.

## Routing — how a router decides where to send a packet

Every router holds a **routing table** keyed by destination network prefix. When a packet arrives the router performs a **longest-prefix match** against its table and forwards the packet out of the interface associated with the winning entry. The packet's destination IP is used — never the source. Layer-2 framing (with MAC addresses) is rewritten at every hop; Layer-3 addresses are preserved end-to-end.

## Routing algorithms

| Family          | How it works                                                                                                                                                          | Example                            | Convergence                                          |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|------------------------------------------------------|
| Distance-vector | Each router tells its neighbours "this is how far I am from every destination"; neighbours add the link cost and re-advertise.                                        | RIP (hop count, max 15)            | Slow; prone to "count to infinity" loops             |
| Link-state      | Every router floods its local link information to every other router, so each builds a full map of the network, then runs Dijkstra's shortest-path algorithm locally. | OSPF, IS-IS                        | Fast; higher memory and CPU per router               |
| Path-vector     | Advertises the <i>entire path</i> (sequence of AS numbers) rather than a scalar distance, for loop avoidance between networks.                                        | BGP — the protocol of the internet | Policy-driven; convergence can take minutes globally |

## Gateway routers

A **gateway** is a router that sits at the boundary between two networks using different protocols or administration domains. Your home router is a gateway between your LAN (RFC1918 private addressing) and your ISP (the public internet). Inside a large corporation, a **border router** peers with the ISP via BGP and advertises the corporation's public prefixes.

**Router** — a Layer-3 device that forwards packets between different IP networks using destination-IP longest-prefix match against its routing table. It terminates each Layer-2 frame and builds a new one for the outgoing link. **Switch** — a Layer-2 device that forwards frames within a single network using destination-MAC lookup against its MAC-address table.

## 4.9.7 Transmission Media

| Medium                                      | Typical bandwidth                      | Attenuation              | Cost            | Notes                                                                                    |
|---------------------------------------------|----------------------------------------|--------------------------|-----------------|------------------------------------------------------------------------------------------|
| Unshielded twisted pair (UTP) — Cat 5e/6/6a | 1–10 Gbps up to 100 m                  | Moderate                 | Cheap           | Twist cancels crosstalk; the universal modern LAN cable (RJ45)                           |
| Shielded twisted pair (STP)                 | As UTP                                 | Lower (better shielding) | Higher than UTP | For electrically noisy environments (factory floors)                                     |
| Coaxial                                     | Up to several Gbps                     | Low for its frequency    | Moderate        | Cable TV, legacy 10BASE2/5 Ethernet, DOCSIS last-mile                                    |
| Multi-mode fibre (MMF)                      | 10–100 Gbps up to a few hundred metres | Higher than SMF          | Lower than SMF  | Short-reach data centres; uses cheap VCSEL light sources; modal dispersion limits length |

|                           |                                          |                                  |                               |                                                               |
|---------------------------|------------------------------------------|----------------------------------|-------------------------------|---------------------------------------------------------------|
| Single-mode fibre (SMF)   | 100 Gbps+, tens to hundreds of km        | Very low (~0.2 dB/km at 1550 nm) | Higher                        | Long-haul telco, transatlantic submarine cables; laser source |
| Radio (Wi-Fi, cellular)   | Up to multi-Gbps (Wi-Fi 6E/7, 5G mmWave) | Distance, walls, interference    | Low deployment, high spectrum | Shared medium; security is a design concern from the start    |
| Microwave (line-of-sight) | Gbps                                     | Rain fade; needs LoS             | Tower/dish capital            | Backhaul between cell sites; trading-firm low-latency links   |
| Infrared                  | Low (legacy IrDA); high for FSO          | Any obstruction                  | Low for IrDA                  | Remote controls, point-to-point free-space optics             |

**Security differences:** copper and fibre can be tapped physically (fibre tapping is harder but has been done by state-level actors); wireless is the easiest to intercept passively because the signal literally propagates through air — encryption is therefore not optional on Wi-Fi.

## 4.9.8 The Client-Server Model

Under the **client-server** model one or more client machines initiate requests to a dedicated server that responds. The web is the canonical example: a browser (client) issues an HTTP request; a web server returns the resource. Servers are usually always-on, rack-mounted, and identified by a stable DNS name; clients are ephemeral and may come and go.

Contrast with **peer-to-peer (P2P)**, where every node acts as both client and server, sharing resources directly without a central coordinator. Examples: BitTorrent for file distribution, Bitcoin for the blockchain, early Skype voice. P2P scales with participants (more peers = more capacity) and has no single point of failure, but is harder to administer, harder to secure, and illegal-content distribution has been a recurrent problem.

## Thin vs thick clients

**Thick (fat) client** — does most of the processing locally; the server mainly stores/syncs data. Examples: a desktop Office install, a native mobile app that works offline. **Thin client** — the client is essentially a display and input layer; processing runs on the server. Examples: a browser rendering a cloud spreadsheet, RDP into a corporate desktop, a Chromebook.

| Dimension              | Thin client                              | Thick client                                       |
|------------------------|------------------------------------------|----------------------------------------------------|
| Processing             | Server-side                              | Client-side                                        |
| Hardware cost (client) | Low — any browser device                 | Higher — needs decent CPU/RAM                      |
| Network dependence     | Total — dies offline                     | Partial — can work offline                         |
| Update/deployment      | One place (the server)                   | Every client must update                           |
| Security               | Data stays on server — easier to control | Data replicated to client — lose laptop, lose data |

## Load balancing

A **load balancer** sits in front of a pool of servers and distributes incoming requests across them — by round-robin, least-connections, or weighted algorithms. Modern implementations (HAProxy, nginx, AWS ELB) additionally perform health-checking (pull a dead server out of rotation automatically), TLS termination, and sticky sessions (keep a user pinned to one backend if state demands it).

## 4.9.9 / 4.9.11 Application-Layer Protocols

### HTTP and HTTPS

HTTP is a request-response text protocol running over TCP port 80. HTTPS is HTTP inside a TLS tunnel on TCP port 443. A minimal request:

```
GET /products/42 HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html,application/xhtml+xml
Accept-Language: en-GB
Cookie: sessionid=abc123
Connection: keep-alive
```

A minimal response:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 1324
Cache-Control: max-age=3600
Set-Cookie: sessionid=abc123; Secure; HttpOnly

<!DOCTYPE html><html>...</html>
```

### Methods:

- **GET** — retrieve a resource. Idempotent; safe; parameters go in the URL query string.
- **POST** — submit data to be processed (create a new resource, process a form). Not idempotent.
- **PUT** — create or replace the resource at a known URL. Idempotent.
- **DELETE** — remove the resource. Idempotent.
- **HEAD** — like GET but returns only headers; check metadata without downloading body.
- **PATCH** — partially update a resource.

**Status codes** come in families:

- **1xx** informational (100 Continue).
- **2xx** success — **200 OK**, 201 Created, 204 No Content.
- **3xx** redirection — **301 Moved Permanently**, 302 Found, 304 Not Modified.
- **4xx** client error — 400 Bad Request, 401 Unauthorised, 403 Forbidden, **404 Not Found**, 429 Too Many Requests.

- **5xx** server error — **500 Internal Server Error**, 502 Bad Gateway, 503 Service Unavailable.

## Common ports

| Port      | Protocol                | Use                                                      |
|-----------|-------------------------|----------------------------------------------------------|
| 20, 21    | FTP                     | File transfer (data, control)                            |
| 22        | SSH / SCP / SFTP        | Secure shell and secure file transfer                    |
| 23        | Telnet                  | Unencrypted remote login (obsolete)                      |
| 25        | SMTP                    | Send email server-to-server                              |
| 53        | DNS                     | Name resolution (UDP primarily, TCP for large responses) |
| 80        | HTTP                    | Unencrypted web                                          |
| 110       | POP3                    | Retrieve mail (download-and-delete)                      |
| 143       | IMAP                    | Retrieve mail (leave on server, sync across devices)     |
| 443       | HTTPS                   | HTTP over TLS                                            |
| 465 / 587 | SMTPS / SMTP-submission | Send mail from a client (authenticated)                  |
| 993       | IMAPS                   | IMAP over TLS                                            |
| 995       | POP3S                   | POP3 over TLS                                            |

## FTP, SMTP, POP3 vs IMAP, SSH

- **FTP** — a two-connection protocol (control on 21, data on 20). Cleartext; has been largely replaced by SFTP (over SSH) or HTTPS file endpoints.
- **SMTP** — Simple Mail Transfer Protocol, the push protocol used between mail servers and between a client and its outgoing server.

- **POP3** — Post Office Protocol v3. Downloads mail from the server to a single client and typically deletes the server copy.
- **IMAP** — Internet Message Access Protocol. Leaves mail on the server; synchronises folder state across multiple devices. Today's dominant retrieval protocol.
- **SSH** — Secure Shell: encrypted remote login, command execution and tunnelling. Uses asymmetric keys for authentication; replaced Telnet/rlogin/rsh.

## TLS handshake (simplified)

TLS 1.2 HANDSHAKE (abbreviated):

```
Client → Server:  ClientHello
                  - TLS version supported
                  - Cipher suites supported
                  - Random nonce (Client Random)
Server → Client:  ServerHello
                  - Chosen cipher suite
                  - Random nonce (Server Random)
                  Certificate (X.509 chain)
                  ServerHelloDone
Client verifies certificate against trusted CA list.
Client → Server:  ClientKeyExchange
                  - pre-master secret encrypted with server's public key
                    (RSA suite) or ECDHE ephemeral key exchange
                  ChangeCipherSpec
                  Finished (encrypted)
Server → Client:  ChangeCipherSpec
                  Finished (encrypted)
Both sides now derive the same symmetric session key from
Client Random + Server Random + pre-master secret, and switch
to symmetric encryption (AES-GCM etc.) for the bulk data.
```

TLS 1.3 (2018) streamlines this into a single round-trip and removes all legacy key-exchange algorithms with known weaknesses. The principle is the same: asymmetric crypto bootstraps trust and agrees a session key; symmetric crypto does the bulk work because it is orders of magnitude faster.

## 4.9.10 The TCP/IP Protocol Stack

**TCP (Transmission Control Protocol)** — a connection-oriented, reliable, ordered, byte-stream transport protocol. Guarantees delivery by sequencing bytes, acknowledging them, and retransmitting anything not acknowledged. The cost is higher overhead and head-of-line blocking.

**UDP (User Datagram Protocol)** — a connectionless, unreliable, datagram transport protocol. No handshake, no retransmission, no ordering. The payoff is low overhead and low latency — ideal for DNS queries, live audio/video, gaming, and anything built on top of QUIC.

### The four layers

| TCP/IP layer          | Role                                                                              | Units of data                    | Example protocols                      |
|-----------------------|-----------------------------------------------------------------------------------|----------------------------------|----------------------------------------|
| Application           | End-user protocols; generates the data to send                                    | Messages / streams               | HTTP, HTTPS, DNS, SMTP, IMAP, FTP, SSH |
| Transport             | End-to-end delivery; reliability (TCP) or speed (UDP); multiplexes by port number | Segments (TCP) / datagrams (UDP) | TCP, UDP, QUIC (over UDP)              |
| Internet (Network)    | Addressing and routing across networks                                            | Packets                          | IP (v4/v6), ICMP, ARP-adjacent         |
| Network Access (Link) | Framing and bit-level delivery on a single physical link                          | Frames / bits                    | Ethernet, Wi-Fi (802.11), PPP          |

### Encapsulation and decapsulation

SENDING DOWN THE STACK (encapsulation):

Application: [ HTTP GET /index.html ]

```

Transport:    [ TCP hdr | HTTP GET /index.html ]
Internet:    [ IP hdr  | TCP hdr | HTTP GET ... ]
Link:        [ Eth hdr | IP hdr  | TCP hdr | HTTP GET ... | Eth CRC ]
                                     ↓ bits on wire

```

#### RECEIVING UP THE STACK (decapsulation):

```

Strip Ethernet → give IP packet to IP layer
Strip IP       → give TCP segment to TCP layer
Strip TCP      → give HTTP message to the browser

```

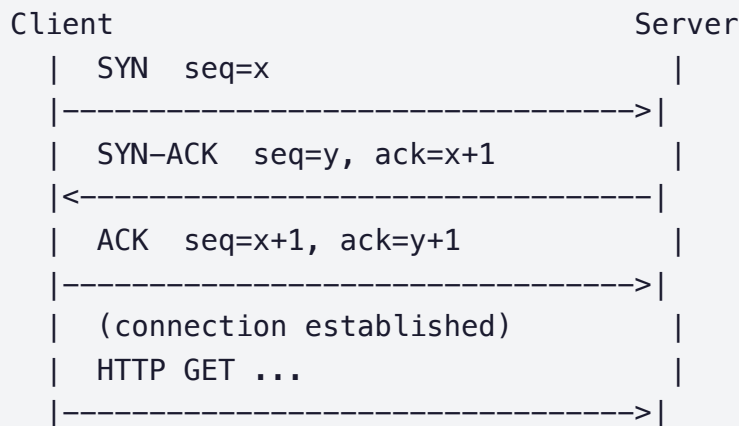
Each layer adds its own header (and optionally trailer) to the payload it receives from the layer above. The receiver performs the reverse — strip off each header and pass the enclosed unit upward. Headers at different layers use different addresses: Layer 2 uses MAC, Layer 3 uses IP, Layer 4 uses port numbers. Crucially, **IP addresses survive every hop across the internet; MAC addresses are rewritten at every router.**

### Brief comparison with OSI 7-layer

| OSI layer                                | TCP/IP equivalent     |
|------------------------------------------|-----------------------|
| 7 Application, 6 Presentation, 5 Session | Application           |
| 4 Transport                              | Transport             |
| 3 Network                                | Internet              |
| 2 Data link, 1 Physical                  | Network Access / Link |

**Exam Tip:** When describing the TCP/IP stack, always give one concrete protocol per layer and one concrete action. "At the application layer HTTP formats the request; at the transport layer TCP opens a connection with a 3-way handshake and numbers the bytes; at the internet layer IP wraps each segment in a packet with source and destination IPs; at the link layer Ethernet adds the MAC addresses of the next hop and a CRC." That is a full-mark answer because every layer has a protocol and a behaviour.

### TCP 3-way handshake



Three segments, one round-trip. Each side learns the other's initial sequence number; from this point on every byte has a sequence number and every received byte is acknowledged cumulatively. Connection teardown is a 4-way FIN/ACK exchange (each direction closes independently).

## 4.9.12 IP Addressing

### IPv4

An IPv4 address is 32 bits, written as four dotted decimal octets: `192.168.1.42`. That gives  $2^{32} \approx 4.3$  billion addresses — fewer than there are humans, and the reason IPv4 exhaustion is a recurring theme. The public-IPv4 free-pool was formally exhausted by IANA in 2011; RIRs have since been allocating from progressively smaller reserves.

### Classful addressing (historical)

Originally IPs were split into fixed classes: Class A (8-bit network, 24-bit host; 126 networks, 16M hosts each), Class B (16/16; 16k networks, 65k hosts), Class C (24/8; 2M networks, 254 hosts). Wasteful — a Class A is absurdly big, a Class C usually too small — and abandoned in 1993 in favour of CIDR.

### CIDR — Classless Inter-Domain Routing

**CIDR notation** — an IP prefix followed by a slash and the number of bits that form the network portion. `192.168.1.0/24` means the first 24 bits are network; the remaining 8 bits are host, giving  $2^8 = 256$  addresses (254 usable, after subtracting network and broadcast addresses).

**Subnet mask** — a 32-bit value with 1s in the network portion and 0s in the host portion; the ANDing of an IP with the mask yields the network address. `/24` is equivalent to `255.255.255.0` ; `/26` is `255.255.255.192` .

### Worked example — how many hosts on `192.168.1.0/24` ?

```

/24 means 24 network bits, 32 - 24 = 8 host bits
Total addresses   = 2^8           = 256
Usable hosts      = 256 - 2       = 254   (minus network and broadcast)
Network address   = 192.168.1.0
Broadcast address = 192.168.1.255
First host        = 192.168.1.1
Last host         = 192.168.1.254
  
```

### Worked example — split `192.168.1.0/24` into `/26` subnets

```

Borrow 26 - 24 = 2 bits from the host portion.
Number of subnets = 2^2 = 4.
Remaining host bits = 32 - 26 = 6 → 2^6 = 64 addresses each,
    of which 62 are usable.
  
```

```

Subnet 0: 192.168.1.0/26      (hosts .1 - .62,    broadcast .63)
Subnet 1: 192.168.1.64/26     (hosts .65 - .126, broadcast .127)
Subnet 2: 192.168.1.128/26    (hosts .129 - .190, broadcast .191)
Subnet 3: 192.168.1.192/26    (hosts .193 - .254, broadcast .255)
  
```

### IP to binary example

```

192 . 168 . 1 . 42
= 11000000 . 10101000 . 00000001 . 00101010
  
```

```
Mask /26 = 11111111 . 11111111 . 11111111 . 11000000
AND      = 11000000 . 10101000 . 00000001 . 00000000
          = 192.168.1.0    ← network address of its /26 subnet
```

## Subnet size reference

| CIDR | Mask            | Addresses | Usable hosts |
|------|-----------------|-----------|--------------|
| /24  | 255.255.255.0   | 256       | 254          |
| /25  | 255.255.255.128 | 128       | 126          |
| /26  | 255.255.255.192 | 64        | 62           |
| /27  | 255.255.255.224 | 32        | 30           |
| /28  | 255.255.255.240 | 16        | 14           |
| /29  | 255.255.255.248 | 8         | 6            |
| /30  | 255.255.255.252 | 4         | 2            |

**Exam Tip:** The single most common CIDR mistake is forgetting to subtract two (network + broadcast) for usable hosts. A /26 has 64 addresses but only 62 hosts. Another trap: a /30 has 4 addresses but only 2 are usable; RFC 3021 allows point-to-point use of both addresses on a /31. In exam conditions, always state the formula " $2^{\text{host bits}} - 2$ " in your working.

## Public vs private (RFC 1918)

| Block                       | CIDR          | Typical use            |
|-----------------------------|---------------|------------------------|
| 10.0.0.0 – 10.255.255.255   | 10.0.0.0/8    | Large enterprise LANs  |
| 172.16.0.0 – 172.31.255.255 | 172.16.0.0/12 | Medium enterprise LANs |

|                                  |                                     |                                  |
|----------------------------------|-------------------------------------|----------------------------------|
| 192.168.0.0 –<br>192.168.255.255 | 192.168.0.0/16                      | Home routers, small offices      |
| 127.0.0.0/8                      | Loopback (127.0.0.1 =<br>localhost) | Local machine only               |
| 169.254.0.0/16                   | Link-local (APIPA)                  | Auto-assigned when DHCP<br>fails |

**NAT (Network Address Translation)** — the technique by which a router rewrites the source IP (and port) of outbound packets to its own public IP, and undoes the mapping on return traffic. This allows many devices on a private-IP LAN to share a single public IPv4 address. NAT delayed IPv4 exhaustion by decades but breaks end-to-end connectivity: inbound connections to a host behind NAT require port forwarding or hole punching.

## IPv4 exhaustion and IPv6

IPv6 addresses are 128 bits —  $2^{128} \approx 3.4 \times 10^{38}$ , enough for roughly  $5 \times 10^{28}$  addresses per human on Earth. Written as eight 16-bit hex groups separated by colons, with double colons compressing runs of zeros:

2001:0db8:85a3:0000:0000:8a2e:0370:7334  
 → 2001:db8:85a3::8a2e:370:7334 (canonical shortened form)

fe80::/10 link-local  
 fc00::/7 unique-local (private, analogous to RFC 1918)  
 2000::/3 global unicast  
 ::1 loopback (analogous to 127.0.0.1)

| Property       | IPv4           | IPv6                |
|----------------|----------------|---------------------|
| Address length | 32 bits        | 128 bits            |
| Notation       | Dotted decimal | Colon-separated hex |

|                    |                            |                                                               |
|--------------------|----------------------------|---------------------------------------------------------------|
| Address space      | ~4.3 billion               | $\sim 3.4 \times 10^{38}$                                     |
| Header             | 20 bytes minimum, variable | 40 bytes fixed, extension headers                             |
| Broadcast          | Yes                        | Replaced by multicast                                         |
| NAT                | Ubiquitous                 | Discouraged — global addressing                               |
| Checksum in header | Yes                        | Removed — trust lower/upper layers                            |
| Auto-configuration | DHCP                       | SLAAC (Stateless Address Autoconfiguration) + optional DHCPv6 |

## 4.9.14 Web Technologies

### Client-side: HTML, CSS, JavaScript

- **HTML** — HyperText Markup Language; declarative structure of a page (headings, paragraphs, forms, tables). The browser's HTML parser builds the DOM (Document Object Model).
- **CSS** — Cascading Style Sheets; declarative presentation (colours, layout, typography). A rule's specificity + source order + `!important` determine who wins. The CSS parser builds the CSSOM, which is merged with the DOM into the render tree.
- **JavaScript** — the web's imperative programming language. Runs in every browser's JS engine (V8, SpiderMonkey, JavaScriptCore), can mutate the DOM, make asynchronous HTTP requests (`fetch`, XHR), and manage state.

### Server-side scripting

Server-side code renders HTML or returns JSON in response to a request. Traditional stacks: PHP (WordPress, early Facebook), Node.js (JavaScript on the server), Python (Django, Flask, FastAPI), Ruby on Rails, Java (Spring), C# (.NET). Modern trend is for the server to expose a JSON API and the browser to render client-side, or for the framework (Next.js, Remix) to do hybrid server-side rendering.

## REST

**REST** (Representational State Transfer) is an architectural style for HTTP APIs: resources are identified by URLs; operations map to HTTP methods; state is transferred as representations (typically JSON). A RESTful endpoint: `GET /api/users/42` returns user 42; `DELETE /api/users/42` removes them; `POST /api/users` creates a new one. REST dominated APIs through the 2010s; alternatives such as GraphQL and gRPC have risen for specific cases.

## AJAX

AJAX (Asynchronous JavaScript And XML — now more often JSON than XML) is the technique of updating a web page without a full reload: the page's JavaScript issues a background HTTP request, receives data, and patches the DOM. Underpinned the Web 2.0 revolution (Gmail 2004, Google Maps 2005); modern `fetch()` is AJAX with a nicer API.

### 4.9.15 MAC Addresses

**MAC address** — a 48-bit Media Access Control address burned into a network interface at manufacture. Written as six colon- or hyphen-separated hex bytes: `00:1A:2B:3C:4D:5E`. The first 3 bytes (OUI — Organisationally Unique Identifier) identify the manufacturer (Cisco, Apple, Dell); the last 3 bytes uniquely identify the interface from that manufacturer, intended to be globally unique.

MAC addresses operate at Layer 2 (the data-link layer). A switch's core job is to maintain a table of *which MAC is on which port* and forward each frame only to the port where the destination MAC lives. If the MAC is unknown the switch floods the frame out of every port except the one it came in on — and learns the sender's MAC from the source field as it goes.

#### MAC vs IP — different layers, different jobs:

| Aspect | MAC address   | IP address         |
|--------|---------------|--------------------|
| Layer  | 2 (Data-link) | 3 (Network)        |
| Length | 48 bits       | 32 (v4) / 128 (v6) |

|                                     |                                  |                    |
|-------------------------------------|----------------------------------|--------------------|
| Scope                               | One local network segment        | Globally routable  |
| Assigned by                         | Hardware manufacturer (IEEE OUI) | Admin / DHCP / ISP |
| Changes when device moves networks? | No                               | Yes                |
| Used by                             | Switches                         | Routers            |

**ARP** (Address Resolution Protocol) glues the two together: when a host wants to send an IP packet on its local network, it must find the MAC of the next hop. It broadcasts "who has IP X?"; the owner replies with its MAC.

**Exam Tip:** A classic trick question: "A packet leaves London for Tokyo. What happens to the addresses?" The source and destination IP stay the same end-to-end (unless NAT intervenes). The source and destination MAC are rewritten at every single router hop. Stating that distinction is often worth 2 marks on its own.

## 4.9.16 Security

### Firewalls

**Firewall** — a device or software that inspects network traffic and permits or denies it according to configured rules. Sits at a boundary between trusted and untrusted networks.

| Type                         | How it works                                                               | Strength                                                                                        |
|------------------------------|----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Packet-filtering (stateless) | Inspects each packet independently against rules on IP, port and protocol. | Fast; no memory; blind to context (e.g. can't tell a SYN-ACK response from an unsolicited one). |

|                                   |                                                                                                                  |                                                                                 |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Stateful inspection               | Tracks active connections in a state table; allows return traffic only for flows that were explicitly initiated. | Dominant modern design.                                                         |
| Application-level gateway (proxy) | Terminates the connection, inspects at the application layer (HTTP, SMTP), then relays.                          | Deep visibility; can block on URL, user, content; slower and protocol-specific. |
| Next-Generation Firewall (NGFW)   | Combines stateful + deep packet inspection + IDS + application identification.                                   | What enterprises buy today.                                                     |

## Intrusion detection (IDS) and prevention (IPS)

An IDS passively monitors traffic and raises alerts on suspicious patterns; an IPS sits inline and can block. Signature-based detection matches traffic against a database of known attack patterns (fast, misses novel attacks). Anomaly-based detection builds a model of normal behaviour and flags deviations (catches novel attacks, more false positives). Snort and Suricata are the open-source canon.

## VPNs

**VPN (Virtual Private Network)** — a secure, encrypted tunnel over an untrusted network that makes two endpoints appear to be on the same private network. Implementations include IPsec (site-to-site), OpenVPN, WireGuard, SSL/TLS VPNs. Purposes: remote-access to a corporate network, protecting traffic on hostile Wi-Fi, bypassing geographic content restrictions.

A VPN **tunnels** one protocol inside another (encrypted) protocol — typically IP packets inside ESP (IPsec) or UDP (WireGuard) — and the payload is encrypted with a symmetric key derived from a Diffie-Hellman exchange. To the outside world only the outer tunnel endpoints and encrypted bytes are visible; the inner IP addresses, ports and payloads are hidden.

## HTTPS and TLS

**TLS (Transport Layer Security)** — the cryptographic protocol that provides confidentiality, integrity and authentication for data transmitted over a TCP connection. Successor to SSL (SSL 3.0 is deprecated). TLS 1.2 is widely deployed; TLS 1.3 (2018) is the modern default. HTTPS is simply HTTP wrapped in TLS.

## Symmetric vs asymmetric cryptography

- **Symmetric** — one shared secret key encrypts and decrypts. Fast (AES at gigabytes per second on modern CPUs with AES-NI), but the key-distribution problem is severe: how do you share a secret before you have a secure channel?
- **Asymmetric (public-key)** — each party has a key pair (public + private). Data encrypted with the public key can only be decrypted with the private key; a signature made with the private key can be verified with the public key. Solves key distribution at the cost of being ~1000× slower than symmetric.

Real systems compose them: use asymmetric crypto (RSA, ECDHE) to *exchange* a session key, then use symmetric crypto (AES) for the bulk of the conversation. This is what TLS does.

## Diffie-Hellman key exchange (intro)

Diffie-Hellman allows two parties to agree on a shared secret over an insecure channel without ever transmitting the secret. They publicly agree on a prime  $p$  and generator  $g$ . Alice picks a private  $a$ , sends  $g^a \bmod p$ ; Bob picks a private  $b$ , sends  $g^b \bmod p$ . Both compute  $g^{ab} \bmod p$  — the shared secret. An eavesdropper who sees  $g^a$  and  $g^b$  cannot feasibly recover  $a$  or  $b$  (discrete-log hardness).

## Digital certificates and PKI

Public-key crypto answers "how do we agree a session key?" but not "is this actually Amazon's public key or an attacker's?". That is answered by **PKI (Public-Key Infrastructure)**:

- A **digital certificate** (X.509 format) binds a public key to a domain name and is *signed by a Certificate Authority (CA)*.
- A **CA** is a trusted third party (Let's Encrypt, DigiCert, GlobalSign). Browsers and operating systems ship with a built-in list of trusted root CAs.

- A **chain of trust**: a website's certificate is usually signed by an intermediate CA, which is signed by a root CA, which is in the browser's trust store. The browser verifies every link in the chain.
- Revocation: CRLs (Certificate Revocation Lists) and OCSP allow a CA to revoke a compromised cert before its expiry.

This is why the green padlock means something: the browser has cryptographically confirmed that the TLS peer possesses the private key corresponding to a certificate signed by a CA the browser trusts, for the domain shown in the URL bar.

### 4.9.17 Wireless Networking Security

| Standard | Year | Encryption                                             | Status                                                                              |
|----------|------|--------------------------------------------------------|-------------------------------------------------------------------------------------|
| WEP      | 1997 | RC4 with 40/104-bit key + 24-bit IV                    | Broken — recoverable key in minutes; never use.                                     |
| WPA      | 2003 | TKIP (RC4 wrapper)                                     | Transitional. Also broken; deprecated.                                              |
| WPA2     | 2004 | AES-CCMP (128-bit)                                     | Still widely deployed. KRACK attack (2017) against the 4-way handshake was patched. |
| WPA3     | 2018 | SAE (Simultaneous Authentication of Equals) + AES-GCMP | Current standard. SAE resists offline dictionary attacks that plagued WPA2-PSK.     |

**SSID (Service Set Identifier)** — the network name broadcast by the AP. Hiding the SSID is security theatre; clients actively probe for named networks and announce them. **MAC filtering** — allow/deny list of MACs permitted to associate. Trivially bypassed because MAC addresses are not authenticated and can be spoofed by software.

A Wi-Fi frame carries more addresses than Ethernet: typically four MAC fields (source, destination, BSSID, optionally next-hop in mesh mode), plus 802.11-specific framing (frame-control, duration, sequence-control) and an FCS. This complexity is why Wi-Fi attacks — deauth floods, evil-twin APs, KRACK — have more surface area than wired attacks.

## 4.9.18 Thin-Client vs Thick-Client (revisited)

Beyond the basic split in 4.9.8, you should know the **economic drivers**. A school running 200 Chromebooks pushes all state to Google and can replace a Chromebook in minutes — the thin-client case. A video-editing studio cannot stream 4K raw footage over the internet every frame and must keep local horsepower — the thick-client case. The correct answer in an exam is almost always *"it depends on the workload and the network"*, with a sentence of justification drawn from bandwidth, latency, data sensitivity, and update-management costs.

## 4.9.19 WebSockets, Streaming, Caching

**WebSocket** — a bidirectional, full-duplex communication channel that runs over a single long-lived TCP connection, established by an HTTP Upgrade handshake. Unlike plain HTTP, either side can push data at any time without the other polling. Used for chat, collaborative editing (Google Docs), live sports scoreboards, trading dashboards.

### WEBSOCKET HANDSHAKE:

```
Client → Server: GET /chat HTTP/1.1
                  Host: example.com
                  Upgrade: websocket
                  Connection: Upgrade
                  Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
                  Sec-WebSocket-Version: 13
Server → Client: HTTP/1.1 101 Switching Protocols
                  Upgrade: websocket
                  Connection: Upgrade
                  Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+x0o=
(connection now speaks WebSocket frames rather than HTTP)
```

**Streaming** — video services (YouTube, Netflix) use HTTP-based adaptive streaming (HLS, DASH): the video is chopped into 2–10 second segments at several bitrates; the client fetches segments over HTTP and switches bitrate based on measured throughput. Live streaming (Twitch) is the same technique with short buffers. Under the hood this is still HTTP requests — which lets CDNs cache segments trivially.

**Caching** happens at many layers: browser cache ( `Cache-Control: max-age=3600` ), CDN edge cache (shared across users in a region), reverse proxy at the origin, and within the origin's database. Cache invalidation is, as the joke goes, one of the two hard problems in computer science.

## End-to-End Worked Example — Visiting

`https://www.example.com`

### Case Study: What actually happens when you press Enter on

`https://www.example.com` .

**1. URL parse.** Browser extracts `scheme= https` , `host= www.example.com` , `port=443` (default), `path= /` .

**2. DNS lookup.** OS stub resolver asks the configured recursive resolver for an A/AAAA record. If cached, one round-trip. If not, recursive walk `root` → `.com` → `example.com`'s authoritative NS, eventually returning `93.184.216.34` . TTL governs re-lookup.

**3. TCP 3-way handshake.** Browser opens a TCP connection to `93.184.216.34` port `443`: `SYN` → `SYN-ACK` → `ACK`. At the internet layer each packet is wrapped in an IP header; at the link layer each IP packet is wrapped in an Ethernet/Wi-Fi frame whose destination MAC is the default gateway (found via ARP) — MACs are rewritten at every hop across the internet, source and destination IPs are not.

**4. TLS handshake.** `ClientHello` (with supported cipher suites and SNI carrying `www.example.com` so a shared-IP server knows which certificate to present) → `ServerHello` + `Certificate` → `key agreement (ECDHE)` → `Finished`. Browser verifies certificate chain against its trusted CA roots and that the domain in the cert matches `www.example.com` . Session keys derived.

**5. HTTP request.** Inside the TLS tunnel: `GET / HTTP/1.1` with `Host`, `User-Agent`, `Accept`, `Cookie` headers.

**6. Server response.** HTTP/1.1 200 OK with HTML body. Browser parses HTML, builds the DOM, fetches CSS/JS/images (often from CDN origins, which repeats steps 2–5 for each new origin), builds the render tree, paints.

**7. Connection reuse / close.** HTTP/1.1 keep-alive or HTTP/2 multiplexing reuses the TCP connection for further requests; eventually TCP FIN/ACK tears it down.

In one keystroke you have used DNS, TCP, TLS, HTTP, HTML, CSS, JavaScript, ARP, Ethernet/Wi-Fi, and IP routing across a dozen autonomous systems. Every layer of the stack in 300 milliseconds.

## Summary — What to Memorise for 4.9

- **Layers.** TCP/IP: Application — Transport — Internet — Link. One protocol and one action per layer.
- **Encapsulation.** Every layer wraps the one above it with its own header; destination MAC is rewritten at every router, destination IP survives end-to-end.
- **Switching.** Packet-switched = datagrams share links statistically; circuit-switched = reserved end-to-end; the internet chose the former.
- **CIDR arithmetic.** Usable hosts =  $2^{32-\text{prefix}} - 2$ . Practise splitting /24 into /26, /27, /28.
- **DNS.** Recursive resolver walks root → TLD → authoritative; caches with TTL; record types A/AAAA/MX/CNAME/NS/TXT.
- **TCP 3-way handshake** SYN / SYN-ACK / ACK; reliability via sequence numbers and ACKs; UDP has none of this.
- **TLS.** Asymmetric crypto bootstraps a symmetric session key; certificate + CA chain binds the key to the domain.
- **Wi-Fi security.** WEP broken; WPA broken; WPA2 with strong passphrase is acceptable; WPA3 is the modern standard.
- **Ports.** 22 SSH, 25 SMTP, 53 DNS, 80 HTTP, 110 POP3, 143 IMAP, 443 HTTPS.
- **IPv4 vs IPv6.** 32 vs 128 bits; dotted decimal vs colon hex; NAT vs native; exhaustion is the "why" of IPv6.

Topic 4.9 is the one place in the specification where breadth beats any single deep insight. A candidate who can draw the TCP/IP stack on demand, walk a packet from browser to web server and back, and compute a subnet calculation without panicking, has 80% of the available marks already locked in. The remaining 20% is the application of protocols you have never heard of before, to scenarios you have never seen — but the layers are always the same, and that is what to rehearse.